



南開大學
Nankai University

计算机学院
计算机系统设计实验报告

PA2 实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 4 月 28 日

目录

1 PA2 实验概述	2
2 PA2-1	2
2.1 公共基础	2
2.2 sub 指令	4
2.3 xor 指令	6
2.4 push 指令	7
2.5 pop 指令	8
2.6 call 指令	9
2.7 ret 指令	10
2.8 nop 指令	11
2.9 运行第一个客户程序	12
3 PA2-2	12
3.1 现代指令系统	12
3.1.1 公共基础	12
3.1.2 leave	14
3.1.3 cld/cdq	15
3.1.4 add	15
3.1.5 inc	16
3.1.6 dec	18
3.1.7 cmp	19
3.1.8 neg	19
3.1.9 not	20
3.1.10 and	21
3.1.11 or	22
3.1.12 sal/shl	22
3.1.13 shr	24
3.1.14 sar	25
3.1.15 test	26
3.2 Differential Testing	27
3.3 一键回归测试	29
4 PA2-3	29
4.1 输入输出	29
4.2 时钟	31
4.3 键盘	33
4.4 VGA	34
5 思考题与必答题	37
5.1 思考题	37
5.2 必答题	41

1 PA2 实验概述

本次 PA2 内容大致包括三个部分：

- 了解指令执行原理，编写一些简单的指令，熟悉实现新指令的基本步骤，在 nemu 中运行简单程序 hello world。
- 在第一阶段基础上，完善现代指令系统，学习了解 AM 基本原理，完善实现 Differential Testing。
- 学习并完善 I/O 设备以及 CPU 对输入输出设备的控制。

2 PA2-1

PA2 第一阶段的任务是让 NEMU 能够执行 dummy 等最基础的 AM 程序。为了完成这一目标，需要补全若干 i386 指令的译码入口和执行语义。本阶段主要实现或接入的指令包括：sub、xor、push、pop、call、ret、nop。

实现一条指令时，主要完成两类工作。第一是在 nemu/src/cpu/exec/exec.c 的 opcode_table 或分组表中填入对应的译码函数、执行函数和操作数宽度；第二是在对应的执行文件中使用 RTL 实现该指令的语义。对于会影响标志位或栈的指令，还需要补充 EFLAGS、栈操作和有符号立即数译码等公共基础代码。

2.1 公共基础

EFLAGS 的保存与初始化 sub 和 xor 都会影响 EFLAGS。因此需要先在 nemu/include/cpu/reg.h 中为 CPU_state 增加 EFLAGS 结构。

本实验中定义了如下标志位：

```

1  typedef union {
2      struct {
3          uint32_t CF : 1;
4          uint32_t   : 1;
5          uint32_t   : 4;
6          uint32_t ZF : 1;
7          uint32_t SF : 1;
8          uint32_t   : 1;
9          uint32_t IF : 1;
10         uint32_t   : 1;
11         uint32_t OF : 1;
12         uint32_t   : 20;
13     };
14     uint32_t val;
15 } EFLAGS;
16
```

其中本阶段主要使用 CF、ZF、SF 和 OF。CPU_state 中增加 EFLAGS eflags 后，执行函数就可以通过 cpu.eflags 保存和读取标志位。

在 nemu/src/monitor/monitor.c 的 restart() 中，初始化：

```
1  cpu.eflags.val = 0x00000002;
```

i386 中 EFLAGS 的 bit 1 是保留位，通常保持为 1，因此初始化为 0x2。

RTL 标志位和栈操作支持 在 nemu/include/cpu/rtl.h 中，补充了标志位读写接口：

```
1  make_rtl_setget_eflags(CF)
2  make_rtl_setget_eflags(OF)
3  make_rtl_setget_eflags(ZF)
4  make_rtl_setget_eflags(SF)
```

该宏会生成 rtl_set_CF()、rtl_get_CF() 等函数。执行函数不直接修改位域，而是通过 RTL 接口设置标志位。

同时实现了 ZF 和 SF 的更新函数：

```
1  static inline void rtl_update_ZF(const rtlreg_t* result, int width) {
2      // eflags.ZF <- is_zero(result[width * 8 - 1 .. 0])
3      uint32_t mask = (width == 4 ? 0xffffffffu : ((1u << (width * 8)) - 1));
4      cpu.eflags.ZF = ((*result & mask) == 0);
5  }
6
7  static inline void rtl_update_SF(const rtlreg_t* result, int width) {
8      // eflags.SF <- is_sign(result[width * 8 - 1 .. 0])
9      cpu.eflags.SF = ((*result >> (width * 8 - 1)) & 0x1);
10 }
11
12 static inline void rtl_update_ZFSF(const rtlreg_t* result, int width) {
13     rtl_update_ZF(result, width);
14     rtl_update_SF(result, width);
15 }
16
```

这里根据操作数宽度判断结果是否为 0，以及结果最高位是否为 1。这样 sub、xor 等指令可以直接调用 rtl_update_ZFSF() 完成公共标志位更新。

栈相关指令依赖 rtl_push() 和 rtl_pop()：

```
1  static inline void rtl_push(const rtlreg_t* src1) {
2      rtlreg_t addr = cpu.esp - 4;
3      rtl_sr_l(R_ESP, &addr);
4      rtl_sm(&addr, 4, src1);
5  }
6
7  static inline void rtl_pop(rtlreg_t* dest) {
8      rtlreg_t addr = cpu.esp;
9      rtl_lm(dest, &addr, 4);
10     addr += 4;
```

```

11     rtl_sr_l(R_ESP, &addr);
12 }
13

```

rtl_push() 对应 i386 中 32 位栈的压栈过程：先将 ESP 减 4，再写入内存；rtl_pop() 对应出栈过程：先从 [ESP] 读出数据，再将 ESP 加 4。push、pop、call 和 ret 都复用了这两个 RTL 函数。

有符号立即数译码 call rel32 的跳转目标是相对地址，需要将立即数字段解释为有符号数。因此在 nemu/src/cpu/decode/decode.c 中实现了 decode_op_SI()：

```

1     switch (op->width) {
2         case 1: op->siml = (int8_t)instr_fetch(eip, 1); break;
3         case 4: op->siml = (int32_t)instr_fetch(eip, 4); break;
4         default: assert(0);
5     }
6
7     rtl_li(&op->val, op->siml);
8

```

随后 decode_J() 使用该有符号立即数计算控制转移目标：

```

1     decode_op_SI(eip, id_dest, false);
2     decoding.jump_eip = id_dest->siml + *eip;
3

```

这里的 *eip 已经指向当前指令之后的顺序地址，因此 siml + *eip 就是 i386 相对跳转或相对调用的目标地址。

2.2 sub 指令

i386 手册 Operation sub 的功能是用目的操作数减去源操作数，并将结果写回目的操作数。i386 手册中的 Operation 可以概括为：

```

1     IF SRC is a byte and DEST is a word or dword
2     THEN DEST = DEST - SignExtend(SRC);
3     ELSE DEST <- DEST - SRC;
4     FI;
5

```

执行后会根据结果更新标志位：

```

1     OF, SF, ZF, AF, PF, and CF as described in Appendix C
2

```

其中 CF 反映无符号减法是否发生借位，OF 反映有符号减法是否发生溢出。

opcode 表接入 在 nemu/src/cpu/exec/exec.c 中，补充了 sub 的普通 opcode：

```

1     /* 0x28 */ IDEXW(G2E, sub, 1), IDEX(G2E, sub), IDEXW(E2G, sub, 1), IDEX(E2G, sub),

```

```
/* 0x2c */ IDEXW(I2a, sub, 1), IDEX(I2a, sub), EMPTY, EMPTY,
```

同时, sub r/m, imm 属于 group 1 指令, 扩展 opcode 为 /5, 因此在 gp1 中填入:

```
make_group(gp1,
EX(add), EX(or), EX(adc), EX(sbb),
EX(and), EX(sub), EX(xor), EX(cmp))
```

这样 0x80 /5、0x81 /5、0x83 /5 都会进入 exec_sub()。

执行函数实现 sub 的执行函数位于 nemu/src/cpu/exec/arith.c:

```
make_EHelper(sub) {
    rtl_sub(&t2, &id_dest->val, &id_src->val);
    operand_write(id_dest, &t2);

    rtl_update_ZFSF(&t2, id_dest->width);

    rtl_sltu(&t0, &id_dest->val, &id_src->val);
    rtl_set_CF(&t0);

    rtl_xor(&t0, &id_dest->val, &id_src->val);
    rtl_xor(&t1, &id_dest->val, &t2);
    rtl_and(&t0, &t0, &t1);
    rtl_msb(&t0, &t0, id_dest->width);
    rtl_set_OF(&t0);

    print_asm_template2(sub);
}
```

CF 和 OF 的更新可以写成一个辅助函数 set_sub_flags() 方便后面复用:

```
static inline void set_sub_flags(uint32_t dest, uint32_t src, uint32_t result, int width) {
    rtl_li(&t0, dest < src);
    rtl_set_CF(&t0);
    rtl_li(&t0, (((dest ^ src) & (dest ^ result) & sign_mask(width)) != 0));
    rtl_set_OF(&t0);
}
```

CF 的判断依据是无符号减法中 $dest < src$ 时发生借位。OF 的判断依据是: 源操作数和目的操作数符号不同, 且结果符号和目的操作数符号不同, 此时发生有符号溢出。这里使用 `sign_mask(width)` 取对应宽度的符号位。

2.3 xor 指令

i386 手册 Operation xor 对两个操作数按位异或，并将结果写回目的操作数。i386 手册中的 Operation 可以概括为：

```
1  DEST <- LeftSRC XOR RightSRC
2  CF <- 0
3  OF <- 0
4
```

xor 是逻辑指令，不产生进位和有符号溢出，因此 CF 和 OF 被清零。ZF 和 SF 根据结果更新。

opcode 表接入 在 nemu/src/cpu/exec/exec.c 中，补充了 xor 的普通 opcode：

```
1  /* 0x30 */ IDEXW(G2E, xor, 1), IDEX(G2E, xor), IDEXW(E2G, xor, 1), IDEX(E2G, xor),
2  /* 0x34 */ IDEXW(I2a, xor, 1), IDEX(I2a, xor), EMPTY, EMPTY,
3
```

这些表项分别支持寄存器/内存之间的异或，以及累加器和立即数之间的异或。

xor r/m, imm 同样属于 group 1 指令，扩展 opcode 为 /6，因此 gp1 中第 6 项填入 EX(xor)：

```
1  make_group(gp1,
2      EX(add), EX(or), EX adc), EX(sbb),
3      EX(and), EX(sub), EX(xor), EX(cmp))
4
```

执行函数实现 xor 的执行函数位于 nemu/src/cpu/exec/logic.c：

```
1  make_EHelper(xor) {
2      rtl_xor(&t2, &id_dest->val, &id_src->val);
3      operand_write(id_dest, &t2);
4
5      rtl_update_ZFSF(&t2, id_dest->width);
6      rtl_set_CF(&tzero);
7      rtl_set_OF(&tzero);
8
9      print_asm_template2(xor);
10 }
11
```

实现过程如下：

- 通过 rtl_xor() 计算 id_dest->val ^ id_src->val，结果保存到临时寄存器 t2。
- 调用 operand_write() 将 t2 写回目的操作数。
- 调用 rtl_update_ZFSF() 根据结果更新 ZF 和 SF。
- 使用 rtl_set_CF(&tzero) 和 rtl_set_OF(&tzero) 将 CF、OF 清零。

rtl_xor() 由 rtl.h 中的 make_rtl_arith_logic(xor) 生成，本质上执行 C 层面的按位异或。这样执行函数只需要表达指令语义，不需要直接处理寄存器和内存写回细节。

2.4 push 指令

i386 手册 Operation push 将源操作数压入栈。在 32 位栈地址大小下，i386 手册中的 Operation 可以概括为：

```

1  IF StackAddrSize = 16
2  THEN
3      IF OperandSize = 16 THEN
4          SP <- SP - 2;
5          (SS:SP) <- (SOURCE); (* word assignment *)
6      ELSE
7          SP <- SP - 4;
8          (SS:SP) <- (SOURCE); (* dword assignment *)
9      FI;
10 ELSE (* StackAddrSize = 32 *)
11     IF OperandSize = 16
12     THEN
13         ESP <- ESP - 2;
14         (SS:ESP) <- (SOURCE); (* word assignment *)
15     ELSE
16         ESP <- ESP - 4;
17         (SS:ESP) <- (SOURCE); (* dword assignment *)
18     FI;
19 FI;
20
```

如果操作数宽度为 16 位，则栈指针减少 2；本实验主要运行在 32 位模式下，因此实现中按 4 字节栈操作处理。push 不修改算术标志位。

opcode 表接入 在 nemu/src/cpu/exec/exec.c 中，为 push r32 填写了 0x50 到 0x57：

```

1  /* 0x50 */ IDEXW(r, push, 4), IDEXW(r, push, 4), IDEXW(r, push, 4), IDEXW(r, push, 4),
2  /* 0x54 */ IDEXW(r, push, 4), IDEXW(r, push, 4), IDEXW(r, push, 4), IDEXW(r, push, 4),
3
```

这里使用 decode_r，寄存器编号由 opcode 低 3 位给出，例如 0x50 是 push eax，0x53 是 push ebx。

立即数压栈也接入了同一个执行函数：

```

1  /* 0x68 */ IDEX(push_SI, push), ... IDEXW(push_SI, push, 1), ...
2
```

其中 0x68 是 push imm16/32，0x6a 是 push imm8。push_SI 会将立即数按有符号数读入，符合 i386 中 push imm8 需要符号扩展的语义。

执行函数实现 push 的执行函数位于 nemu/src/cpu/exec/data-mov.c：

```

1  make_EHelper(push) {
2      rtl_push(&id_dest->val);
3
4      print_asm_template1(push);
5  }
6

```

这里 `id_dest` 表示被压栈的源操作数。虽然名字是 `dest`，但对一元指令而言，框架统一将译码得到的操作数放在 `id_dest` 中。

真正的栈操作由 `rtl_push()` 完成，见前文。

2.5 pop 指令

i386 手册 Operation `pop` 从栈顶弹出数据到目的操作数。在 32 位栈地址大小下，i386 手册中的 Operation 可以概括为：

```

1  IF StackAddrSize = 16
2  THEN
3      IF OperandSize = 16
4      THEN
5          DEST <- (SS:SP); (* copy a word *)
6          SP <- SP + 2;
7      ELSE (* OperandSize = 32 *)
8          DEST <- (SS:SP); (* copy a dword *)
9          SP <- SP + 4;
10     FI;
11 ELSE (* StackAddrSize = 32 * )
12     IF OperandSize = 16
13     THEN
14         DEST <- (SS:ESP); (* copy a word *)
15         ESP <- ESP + 2;
16     ELSE (* OperandSize = 32 *)
17         DEST <- (SS:ESP); (* copy a dword *)
18         ESP <- ESP + 4;
19     FI;
20 FI;
21

```

如果目的操作数是内存，处理器先从栈顶取值，再更新栈指针。`pop` 通常不修改算术标志位。

opcode 表接入 在 `nemu/src/cpu/exec/exec.c` 中，为 `pop r32` 填写了 `0x58` 到 `0x5f`：

```

1  /* 0x58 */ IDEXW(r, pop, 4), IDEXW(r, pop, 4), IDEXW(r, pop, 4), IDEXW(r, pop, 4),
2  /* 0x5c */ IDEXW(r, pop, 4), IDEXW(r, pop, 4), IDEXW(r, pop, 4), IDEXW(r, pop, 4),
3

```

此外，`0x8f /0` 形式的 `pop r/m32` 使用 `IDEX(E, pop)` 接入：

```
1  /* 0x8c */ EMPTY, IDEX(lea_M2G, lea), EMPTY, IDEX(E, pop),
```

decode_E 会读取 ModR/M，得到目的寄存器或目的内存地址。

执行函数实现 pop 的执行函数位于 nemu/src/cpu/exec/data-mov.c:

```
1  make_EHelper(pop) {
2      rtl_pop(&t0);
3      operand_write(id_dest, &t0);
4
5      print_asm_template1(pop);
6  }
```

实现过程如下：

- 调用 rtl_pop(&t0) 从栈顶取出 4 字节数据到临时寄存器 t0。
- 调用 operand_write(id_dest, &t0) 将取出的值写入目的操作数。

rtl_pop() 的实现见前文。

2.6 call 指令

i386 手册 Operation i386 手册中 near call 的 Operation 可以概括为：

```
1  IF rel16 or rel32 type of call
2  THEN (* near relative call *)
3      IF OperandSize = 16
4      THEN
5          Push(IP);
6          EIP <- (EIP + rel16) AND 0000FFFFH;
7      ELSE (* OperandSize = 32 *)
8          Push(EIP);
9          EIP <- EIP + rel32;
10     FI;
11 FI;
12
13 IF r/m16 or r/m32 type of call
14 THEN (* near absolute call *)
15     IF OperandSize = 16
16     THEN
17         Push(IP);
18         EIP <- [r/m16] AND 0000FFFFH;
19     ELSE (* OperandSize = 32 *)
20         Push(EIP);
21         EIP <- [r/m32];
22     FI;
23 FI;
```

24

其中被压栈的 EIP 是 call 指令之后下一条指令的地址，也就是返回地址。DEST 是指令中的相对位移，按有符号数解释。

opcode 表和译码 在 nemu/src/cpu/exec/exec.c 中，call rel32 对应 opcode 0xe8:

```
1 /* 0xe8 */ IDEXW(J, call, 4), IDEX(J, jmp), EMPTY, IDEXW(J, jmp, 1),
2
```

这里使用 decode_J 译码，宽度为 4，表示读取 32 位相对位移。

decode_J() 位于 nemu/src/cpu/decode/decode.c:

```
1 make_DHelper(J) {
2     decode_op_SI(eip, id_dest, false);
3     decoding.jump_eip = id_dest->simmm + *eip;
4 }
5
```

decode_op_SI() 将立即数字段解释为有符号数。由于译码完成后 *eip 已经指向下一条指令，所以 id_dest->simmm + *eip 就是 call 的目标地址。

执行函数实现 call 的执行函数位于 nemu/src/cpu/exec/control.c:

```
1 make_EHelper(call) {
2     // the target address is calculated at the decode stage
3     rtlreg_t ret_addr = *eip;
4     rtl_push(&ret_addr);
5     decoding.is_jump = 1;
6
7     print_asm("call %x", decoding.jump_eip);
8 }
9
```

实现过程如下:

- ret_addr = *eip 保存顺序执行地址。此时 *eip 已经是 call 指令之后的地址，即返回地址。
- 调用 rtl_push(&ret_addr) 将返回地址压入栈。
- 设置 decoding.is_jump = 1，通知执行框架本条指令结束后不要使用顺序地址，而是跳转到 decoding.jump_eip。

最终跳转动作由 exec.c 中的 update_eip() 完成。

2.7 ret 指令

i386 手册 Operation .

```
1 IF instruction = near RET
```

```

2 THEN;
3     IF OperandSize = 16
4     THEN
5         IP <- Pop();
6         EIP <- EIP AND 0000FFFFH;
7     ELSE (* OperandSize = 32 *)
8         EIP <- Pop();
9     FI;
10    IF instruction has immediate operand THEN eSP <- eSP + imm16; FI;
11    FI;
12

```

也就是从当前栈顶弹出 4 字节返回地址，并将其装入 EIP。ret 通常和 call 配对使用，call 压入返回地址，ret 从栈中取回该地址。

opcode 表接入 在 nemu/src/cpu/exec/exec.c 中，ret 对应 opcode 0xc3:

```

1 /* 0xc0 */ IDEXW(gp2_Ib2E, gp2, 1), IDEX(gp2_Ib2E, gp2), EMPTY, EX(ret),
2

```

0xc3 不需要额外操作数，因此使用 EX(ret)，即没有译码函数，直接进入 exec_ret()。

执行函数实现 ret 的执行函数位于 nemu/src/cpu/exec/control.c:

```

1 make_EHelper(ret) {
2     rtl_pop(&t0);
3     decoding.jump_eip = t0;
4     decoding.is_jump = 1;
5
6     print_asm("ret");
7 }
8

```

实现过程如下:

- 调用 rtl_pop(&t0) 从栈顶取出返回地址，同时将 ESP 加 4。
- 将取出的返回地址保存到 decoding.jump_eip。
- 设置 decoding.is_jump = 1，让统一的 update_eip() 将 cpu.eip 更新为返回地址。

该实现对应 i386 手册中的 EIP <- Pop()。由于 rtl_pop() 已经完成了读取栈顶和更新 ESP 的操作，ret 执行函数只需要把弹出的值作为跳转目标。

2.8 nop 指令

nop 是空操作指令，该指令不修改寄存器、不访问内存、不修改 EFLAGS，只让 CPU 继续执行下一条指令。

opcode 表接入 `nop` 的执行函数在框架中已经存在，位于 `nemu/src/cpu/exec/special.c`，只需要在 `nemu/src/cpu/exec/exec.c` 中将 `opcode 0x90` 接入：

```
1  /* 0x90 */ EX(nop), EMPTY, EMPTY, EMPTY,
```

```
2
```

2.9 运行第一个客户程序

完成以上指令后，在 `nexus-am/tests/cputest` 文件夹下运行 `make ARCH=x86-nemu ALL=dummy run` 命令：

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 08:47:08, Apr 27 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100052

(nemu) q
dummy
```

`dummy.c` 运行成功。

3 PA2-2

PA2 第二阶段的目标是在第一阶段已有基础指令之上，继续补全 `dummy` 之后的 AM 测试程序和 C 程序运行所需的 i386 常用算术、逻辑、移位和栈帧维护指令。这些指令的实现仍然遵循 NEMU 的 `decode + execute` 框架。`nemu/src/cpu/exec/exec.c` 中的 `opcode_table` 负责把 `opcode` 映射到译码 helper 和执行 helper；`make_EHelper()` 负责根据译码得到的 `id_dest`、`id_src` 等操作数完成指令语义。

3.1 现代指令系统

3.1.1 公共基础

group opcode 本阶段很多指令不是由 `opcode` 单独决定，而是由 `ModR/M` 字节中的 `reg/opcode` 扩展字段决定。NEMU 使用 `make_group()` 建立 `group` 表：

```
1  #define make_group(name, item0, item1, item2, item3, item4, item5, item6, item7) \
2  static opcode_entry concat(opcode_table_, name) [8] = { \
3      /* 0x00 */ item0, item1, item2, item3, \
4      /* 0x04 */ item4, item5, item6, item7 \
5  }; \
6  static make_EHelper(name) { \
7      idex(eip, &concat(opcode_table_, name)[decoding.ext_opcode]); \
8  }
9
```

相关 `group` 表如下：

```
1  make_group(gp1,
2      EX(add), EX(or), EX(adc), EX(sbb),
```

```

3     EX(and), EX(sub), EX(xor), EX(cmp))
4
5     make_group(gp2,
6         EX(rol), EMPTY, EMPTY, EMPTY,
7         EX(shl), EX(shr), EMPTY, EX(sar))
8
9     make_group(gp3,
10        IDEX(test_I, test), EMPTY, EX(not), EX(neg),
11        EX(mul), EX(imul1), EX(div), EX(idiv))
12
13    make_group(gp4,
14        EX(inc), EX(dec), EMPTY, EMPTY,
15        EMPTY, EMPTY, EMPTY, EMPTY)
16
17    make_group(gp5,
18        EX(inc), EX(dec), EX(call_rm), EMPTY,
19        EX(jmp_rm), EMPTY, EX(push), EMPTY)
20

```

其中 gp1 支持 add/or/and/cmp 的立即数形式, gp2 支持移位指令, gp3 支持 test/not/neg, gp4/gp5 支持 inc/dec。

算术标志位辅助函数 算术指令需要根据操作数宽度截断结果, 并维护 ‘CF’、‘OF’、‘ZF’、‘SF’。arith.c 中实现了公共辅助函数:

```

1     static inline uint32_t mask_width(int width) {
2         return width == 4 ? 0xffffffffu : ((1u << (width * 8)) - 1);
3     }
4
5     static inline uint32_t sign_mask(int width) {
6         return 1u << (width * 8 - 1);
7     }
8
9     static inline void set_add_flags(uint32_t dest, uint32_t src, uint32_t result, int width) {
10        uint32_t mask = mask_width(width);
11        rtl_li(&t0, ((uint64_t)dest + src) > mask);
12        rtl_set_CF(&t0);
13        rtl_li(&t0, ((~(dest ^ src) & (dest ^ result) & sign_mask(width)) != 0));
14        rtl_set_OF(&t0);
15    }
16
17    static inline void set_sub_flags(uint32_t dest, uint32_t src, uint32_t result, int width) {
18        rtl_li(&t0, dest < src);
19        rtl_set_CF(&t0);
20        rtl_li(&t0, (((dest ^ src) & (dest ^ result) & sign_mask(width)) != 0));
21        rtl_set_OF(&t0);
22    }
23

```

mask_width() 用于得到 8/16/32 位结果掩码, sign_mask() 用于得到符号位。set_add_flags() 按加法进位和有符号溢出规则更新 CF/OF; set_sub_flags() 按减法借位和有符号溢出规则更新 CF/OF。ZF/SF 由 rtl_update_ZFSF() 统一更新。

3.1.2 leave

i386 手册 Operation .

```

1  IF StackAddrSize = 16
2  THEN
3      SP <- BP;
4  ELSE (* StackAddrSize = 32 *)
5      ESP <- EBP;
6  FI;
7  IF OperandSize = 16
8  THEN
9      BP <- Pop();
10     ELSE (* OperandSize = 32 *)
11         EBP <- Pop();
12     FI;
13 
```

leave 用于退出过程栈帧。它先将帧指针复制到栈指针, 释放局部变量空间; 再从栈顶弹出旧帧指针, 恢复调用者的 BP/EBP。该指令不影响标志位。

opcode 表接入 .

```

1  /* 0xc8 */ EMPTY, EX(leave), EMPTY, EMPTY,
2 
```

执行函数实现 .

```

1  make_EHelper(leave) {
2      rtl_lr_l(&t0, R_EBP);
3      rtl_sr_l(R_ESP, &t0);
4      rtl_pop(&t0);
5      rtl_sr_l(R_EBP, &t0);
6
7      print_asm("leave");
8  }
9 
```

本实验的 NEMU 运行在 32 位栈模型下, 因此实现采用 EBP/ESP。第一步通过 rtl_lr_l() 读取 EBP 到临时寄存器 t0, 然后通过 rtl_sr_l() 写入 ESP, 对应手册中的 ESP <- EBP。随后调用 rtl_pop(&t0) 从当前栈顶弹出调用者保存的旧 EBP, 最后写回 R_EBP。这正好等价于常见函数尾声中的 mov esp, ebp; pop ebp。

3.1.3 cltd/cdq

i386 手册 Operation .

```

1  IF OperandSize = 16 (* CWD instruction *)
2  THEN
3      IF AX < 0 THEN DX <- 0FFFFH; ELSE DX <- 0; FI;
4  ELSE (* OperandSize = 32, CDQ instruction *)
5      IF EAX < 0 THEN EDX <- 0FFFFFFFFH; ELSE EDX <- 0; FI;
6  FI;
7

```

cdq/cltd 将 EAX 的符号位扩展到整个 EDX，形成 EDX:EAX 的 64 位有符号被除数。它不影响标志位。

opcode 表接入 .

```

1  /* 0x98 */ EX(cwt1), EX(cltd), EMPTY, EMPTY,
2

```

执行函数实现 .

```

1  make_EHelper(cltd) {
2      if (decoding.is_operand_size_16) {
3          rtl_lr_w(&t0, R_AX);
4          rtl_msb(&t1, &t0, 2);
5          rtl_sub(&t1, &tzero, &t1);
6          rtl_sr_w(R_DX, &t1);
7      }
8      else {
9          rtl_lr_l(&t0, R_EAX);
10         rtl_msb(&t1, &t0, 4);
11         rtl_sub(&t1, &tzero, &t1);
12         rtl_sr_l(R_EDX, &t1);
13     }
14
15     print_asm(decoding.is_operand_size_16 ? "cwt1" : "cltd");
16 }
17

```

实现中先判断操作数大小前缀。16 位形式读取 AX，取其最高位后通过 0 - signbit 得到 0x0000 或 0xffff，再写入 DX。32 位形式读取 EAX，同样取符号位并生成 0x00000000 或 0xffffffff，写入 EDX。这样就实现了手册中“把源寄存器最高位扩展到高半部分寄存器”的语义。

3.1.4 add

i386 手册 Operation .

```

1  DEST <- DEST + SRC;

```

add 将两个操作数相加，结果写回目的操作数，并根据结果更新 OF/SF/ZF/AF/CF/PF。本实验中实现了后续程序会用到的 OF/SF/ZF/CF。

opcode 表接入

```
/* 0x00 */ IDEXW(G2E, add, 1), IDEX(G2E, add), IDEXW(E2G, add, 1), IDEX(E2G, add),
/* 0x04 */ IDEXW(I2a, add, 1), IDEX(I2a, add), EMPTY, EMPTY,
```

这部分支持 add r/m, r、add r, r/m 和 add AL/eAX, imm。立即数到 r/m 的形式由 gp1 的 /0 扩展接入：

```
make_group(gp1,
EX(add), EX(or), EX(adc), EX(sbb),
EX(and), EX(sub), EX(xor), EX(cmp))

/* 0x80 */ IDEXW(I2E, gp1, 1), IDEX(I2E, gp1), EMPTY, IDEX(SI2E, gp1),
```

执行函数实现

```
make_EHelper(add) {
    uint32_t mask = mask_width(id_dest->width);
    uint32_t dest = id_dest->val & mask;
    uint32_t src = id_src->val & mask;
    uint32_t result = (dest + src) & mask;

    rtl_li(&t2, result);
    operand_write(id_dest, &t2);

    rtl_update_ZFSF(&t2, id_dest->width);
    set_add_flags(dest, src, result, id_dest->width);

    print_asm_template2(add);
}
```

实现先根据目的操作数宽度生成掩码，避免 8 位或 16 位运算受到高位残留影响。随后计算 dest + src，截断为有效宽度后写回目的操作数。写回由 operand_write() 完成，因此目的操作数是寄存器或内存都可以统一处理。最后更新 ZF/SF，并通过 set_add_flags() 设置加法的进位标志 CF 和有符号溢出标志 OF。

3.1.5 inc

i386 手册 Operation

```
1 DEST <- DEST + 1;
```

inc 将操作数加 1，并更新 OF/SF/ZF/AF/PF，但不改变 CF。

opcode 表接入

```
1 /* 0x40 */ IDEX(r, inc), IDEX(r, inc), IDEX(r, inc), IDEX(r, inc),
2 /* 0x44 */ IDEX(r, inc), IDEX(r, inc), IDEX(r, inc), IDEX(r, inc),
3
```

0x40 到 0x47 是寄存器短编码形式，低 3 位直接编码目的寄存器，因此使用 decode_r()。内存或寄存器通用形式由 group 表接入：

```
1 make_group(gp4,
2     EX(inc), EX(dec), EMPTY, EMPTY,
3     EMPTY, EMPTY, EMPTY, EMPTY)
4
5 make_group(gp5,
6     EX(inc), EX(dec), EX(call_rm), EMPTY,
7     EX(jmp_rm), EMPTY, EX(push), EMPTY)
8
9 /* 0xfc */ EMPTY, EMPTY, IDEXW(E, gp4, 1), IDEX(E, gp5),
10
```

执行函数实现

```
1 make_EHelper(inc) {
2     uint32_t mask = mask_width(id_dest->width);
3     uint32_t dest = id_dest->val & mask;
4     uint32_t result = (dest + 1) & mask;
5
6     rtl_get_CF(&t1);
7     rtl_li(&t2, result);
8     operand_write(id_dest, &t2);
9
10    rtl_update_ZFSF(&t2, id_dest->width);
11    set_add_flags(dest, 1, result, id_dest->width);
12    rtl_set_CF(&t1);
13
14    print_asm_template1(inc);
15 }
16
```

inc 的特殊点是不能改变 CF。实现中先用 rtl_get_CF(&t1) 保存原 CF，再按加 1 运算计算结果并写回。set_add_flags(dest, 1, result, width) 会按加法规则更新 CF/OF，因此最后再用 rtl_set_CF(&t1) 恢复原来的 CF，只保留 OF/ZF/SF 等应被 inc 修改的标志效果。

3.1.6 dec

i386 手册 Operation .

```
DEST <- DEST - 1;
```

dec 将操作数减 1，并更新 OF/SF/ZF/AF/PF，但不改变 CF。

opcode 表接入 .

```
/* 0x48 */ IDEX(r, dec), IDEX(r, dec), IDEX(r, dec), IDEX(r, dec),
/* 0x4c */ IDEX(r, dec), IDEX(r, dec), IDEX(r, dec), IDEX(r, dec),
```

0x48 到 0x4f 是寄存器短编码形式。通用 r/m 形式通过 group 表接入：

```
make_group(gp4,
    EX(inc), EX(dec), EMPTY, EMPTY,
    EMPTY, EMPTY, EMPTY, EMPTY)

make_group(gp5,
    EX(inc), EX(dec), EX(call_rm), EMPTY,
    EX(jmp_rm), EMPTY, EX(push), EMPTY)

/* 0xfc */ EMPTY, EMPTY, IDEXW(E, gp4, 1), IDEX(E, gp5),
```

执行函数实现 .

```
make_EHelper(dec) {
    uint32_t mask = mask_width(id_dest->width);
    uint32_t dest = id_dest->val & mask;
    uint32_t result = (dest - 1) & mask;

    rtl_get_CF(&t1);
    rtl_li(&t2, result);
    operand_write(id_dest, &t2);

    rtl_update_ZFSF(&t2, id_dest->width);
    set_sub_flags(dest, 1, result, id_dest->width);
    rtl_set_CF(&t1);

    print_asm_template1(dec);
}
```

实现与 inc 对称。先保存 CF，再计算 dest - 1 并写回目的操作数。set_sub_flags() 按减法规则计算借位和溢出，最后恢复 CF，保证 dec 不改变进位标志。

3.1.7 cmp

i386 手册 Operation .

```

1 LeftSRC = SignExtend(RightSRC);
2 (* CMP does not store a result; its purpose is to set the flags *)
3

```

cmp 用第一个操作数减去第二个操作数，只更新标志位，不保存结果。它通常配合条件跳转或 setcc 使用。

opcode 表接入 .

```

1 /* 0x38 */ IDEXW(G2E, cmp, 1), IDEX(G2E, cmp), IDEXW(E2G, cmp, 1), IDEX(E2G, cmp),
2 /* 0x3c */ IDEXW(I2a, cmp, 1), IDEX(I2a, cmp), EMPTY, EMPTY,
3

```

这部分支持寄存器/内存互相比以及 AL/eAX 与立即数比较。立即数到 r/m 的形式通过 gp1 /7 接入：

```

1 make_group(gp1,
2     EX(add), EX(or), EX(adc), EX(sbb),
3     EX(and), EX(sub), EX(xor), EX(cmp))
4

```

执行函数实现 .

```

1 make_EHelper(cmp) {
2     uint32_t mask = mask_width(id_dest->width);
3     uint32_t dest = id_dest->val & mask;
4     uint32_t src = id_src->val & mask;
5     uint32_t result = (dest - src) & mask;
6
7     rtl_li(&t2, result);
8     rtl_update_ZFSF(&t2, id_dest->width);
9     set_sub_flags(dest, src, result, id_dest->width);
10
11     print_asm_template2(cmp);
12 }
13

```

cmp 与 sub 的核心计算相同，都是 dest - src。区别是 cmp 不调用 operand_write()，因此不会修改目的操作数。计算出的临时结果只用于更新 ZF/SF/CF/OF。

3.1.8 neg

i386 手册 Operation .

```

1 IF r/m = 0 THEN CF <- 0 ELSE CF <- 1; FI;

```

```
2   r/m <- - r/m;
```

neg 对操作数取二进制补码，相当于用 0 减去该操作数。除原操作数为 0 时 CF 清零外，其余情况 CF 置 1。

opcode 表接入

```
1   make_group(gp3,
2       IDEX(test_I, test), EMPTY, EX(not), EX(neg),
3       EX(mul), EX(imul1), EX(div), EX(idiv))
4
5   /* 0xf4 */ EMPTY, EMPTY, IDEXW(E, gp3, 1), IDEX(E, gp3),
6
```

执行函数实现

```
1   make_EHelper(neg) {
2       uint32_t mask = mask_width(id_dest->width);
3       uint32_t dest = id_dest->val & mask;
4       uint32_t result = (-dest) & mask;
5
6       rtl_li(&t2, result);
7       operand_write(id_dest, &t2);
8
9       rtl_update_ZFSF(&t2, id_dest->width);
10      rtl_li(&t0, dest != 0);
11      rtl_set_CF(&t0);
12      rtl_li(&t0, dest == sign_mask(id_dest->width));
13      rtl_set_OF(&t0);
14
15      print_asm_template1(neg);
16  }
```

实现先将操作数截断到有效宽度，再用 `-dest` 计算补码结果并写回。CF 按手册要求由原操作数是否为 0 决定。OF 只在原操作数为最小负数时置 1，例如 8 位的 0x80、16 位的 0x8000、32 位的 0x80000000，因为这些值取负后无法在同宽度有符号数中表示。

3.1.9 not

i386 手册 Operation

```
1   r/m <- NOT r/m;
```

not 对操作数逐位取反，不影响任何标志位。

opcode 表接入 .

```

1  make_group(gp3,
2      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
3      EX(mul), EX(imul1), EX(div), EX(idiv))
4
5  /* 0xf4 */ EMPTY, EMPTY, IDEXW(E, gp3, 1), IDEX(E, gp3),
6

```

执行函数实现 .

```

1  make_EHelper(not) {
2      rtl_mv(&t2, &id_dest->val);
3      rtl_not(&t2);
4      operand_write(id_dest, &t2);
5
6      print_asm_template1(not);
7  }
8

```

实现将目的操作数复制到 t2，调用 RTL 的 rtl_not() 完成逐位取反，再写回目的操作数。

3.1.10 and

i386 手册 Operation .

```

1  DEST <- DEST AND SRC;
2  CF <- 0;
3  OF <- 0;
4

```

and 对两个操作数逐位与，结果写回目的操作数。逻辑指令会清零 CF 和 OF，并根据结果更新 SF/ZF/PF。

opcode 表接入 .

```

1  /* 0x20 */ IDEXW(G2E, and, 1), IDEX(G2E, and), IDEXW(E2G, and, 1), IDEX(E2G, and),
2  /* 0x24 */ IDEXW(I2a, and, 1), IDEX(I2a, and), EMPTY, EMPTY,
3

```

立即数到 r/m 的形式通过 gp1 /4 接入。

执行函数实现 .

```

1  make_EHelper(and) {
2      rtl_and(&t2, &id_dest->val, &id_src->val);
3      operand_write(id_dest, &t2);
4
5      rtl_update_ZFSF(&t2, id_dest->width);

```

```

6     rtl_set_CF(&tzero);
7     rtl_set_OF(&tzero);
8
9     print_asm_template2(and);
10 }
11

```

实现使用 RTL 的 `rtl_and()` 得到按位与结果，并写回目的操作数。随后按结果更新 ZF/SF，再将 CF 和 OF 清零。

3.1.11 or

i386 手册 Operation .

```

1     DEST <- DEST OR SRC;
2     CF <- 0;
3     OF <- 0;
4

```

`or` 对两个操作数逐位或，结果写回目的操作数。它清零 CF/OF，并根据结果更新 SF/ZF/PF。

opcode 表接入 .

```

1     /* 0x08 */ IDEXW(G2E, or, 1), IDEX(G2E, or), IDEXW(E2G, or, 1), IDEX(E2G, or),
2     /* 0x0c */ IDEXW(I2a, or, 1), IDEX(I2a, or), EMPTY, EX(2byte_esc),
3

```

立即数到 `r/m` 的形式通过 `gp1 /1` 接入。

执行函数实现 .

```

1     make_EHelper(or) {
2         rtl_or(&t2, &id_dest->val, &id_src->val);
3         operand_write(id_dest, &t2);
4
5         rtl_update_ZFSF(&t2, id_dest->width);
6         rtl_set_CF(&tzero);
7         rtl_set_OF(&tzero);
8
9         print_asm_template2(or);
10    }
11

```

实现与 `and` 类似，只是核心运算换成 `rtl_or()`。结果写回后更新 ZF/SF，并清零 CF/OF。

3.1.12 sal/shl

i386 手册 Operation .

```

1     (* COUNT is the second parameter *)

```

```

2  (temp) <- COUNT;
3  WHILE (temp != 0)
4  DO
5      IF instruction is SAL or SHL
6      THEN CF <- high-order bit of r/m;
7      FI;
8      IF instruction = SAL or SHL
9      THEN r/m <- r/m * 2;
10     FI;
11     temp <- temp - 1;
12 OD;
13 (* Determine overflow for the various instructions *)
14 IF COUNT = 1
15 THEN
16     IF instruction is SAL or SHL
17     THEN OF <- high-order bit of r/m != (CF);
18     FI;
19 ELSE OF <- undefined;
20 FI;
21

```

sal/shl 将操作数左移，低位补 0。

opcode 表接入 .

```

1  make_group(gp2,
2      EX(rol), EMPTY, EMPTY, EMPTY,
3      EX(shl), EX(shr), EMPTY, EX(sar))
4
5  /* 0xc0 */ IDEXW(gp2_ib2E, gp2, 1), IDEX(gp2_ib2E, gp2), EMPTY, EX(ret),
6  /* 0xd0 */ IDEXW(gp2_1_E, gp2, 1), IDEX(gp2_1_E, gp2), IDEXW(gp2_c12E, gp2, 1),
7  IDEX(gp2_c12E, gp2),

```

执行函数实现 .

```

1  make_EHelper(shl) {
2      uint32_t count = id_src->val & 0x1f;
3      uint32_t bits = id_dest->width * 8;
4      uint32_t result;
5
6      if (count == 0) goto shl_end;
7
8      if (count >= bits) result = 0;
9      else result = (id_dest->val << count) & logic_mask(id_dest->width);
10
11     rtl_li(&t2, result);
12     operand_write(id_dest, &t2);

```

```

13     rtl_update_ZFSF(&t2, id_dest->width);
14     // unnecessary to update CF and OF in NEMU
15     shl_end:
16     print_asm_template2(shl);
17 }
18

```

实现首先使用 `id_src->val & 0x1f` 得到规定的有效移位次数。如果移位次数为 0，则目的操作数和标志位均保持不变。否则按操作数位宽计算结果：当移位次数大于等于操作数位数时结果为 0；否则左移并用 `logic_mask()` 截断到有效宽度。最后写回目的操作数，并更新 ZF/SF。

3.1.13 shr

i386 手册 Operation .

```

1  (* COUNT is the second parameter *)
2  (temp) <- COUNT;
3  WHILE (temp != 0)
4  DO
5      IF instruction is SAR or SHR
6      THEN CF <- low-order bit of r/m;
7      FI;
8      IF instruction = SHR
9      THEN r/m <- r/m / 2; (* Unsigned divide *);
10     FI;
11     temp <- temp - 1;
12 OD;
13 IF COUNT = 1
14 THEN
15     IF instruction is SHR
16     THEN OF <- high-order bit of operand;
17     FI;
18 ELSE OF <- undefined;
19 FI;
20

```

shr 是逻辑右移，高位补 0，可理解为无符号除以 2 的重复操作。

opcode 表接入 .

```

1 make_group(gp2,
2     EX(rol), EMPTY, EMPTY, EMPTY,
3     EX(shl), EX(shr), EMPTY, EX(sar))
4

```

执行函数实现 .

```

1 make_EHelper(shr) {

```

```

2      uint32_t count = id_src->val & 0x1f;
3      uint32_t bits = id_dest->width * 8;
4      uint32_t result;
5
6      if (count == 0) goto shr_end;
7
8      if (count >= bits) result = 0;
9      else result = (id_dest->val & logic_mask(id_dest->width)) >> count;
10
11     rtl_li(&t2, result);
12     operand_write(id_dest, &t2);
13     rtl_update_ZFSF(&t2, id_dest->width);
14     // unnecessary to update CF and OF in NEMU
15     shr_end:
16     print_asm_template2(shr);
17 }
18

```

实现中先截取移位次数低 5 位，再根据操作数宽度得到有效位数。shr 需要逻辑右移，因此先用 logic_mask() 清除无效高位，再执行无符号右移。移位次数为 0 时不改变目的操作数；移位次数大于等于位宽时结果为 0。结果写回后更新 ZF/SF。

3.1.14 sar

i386 手册 Operation .

```

1      (* COUNT is the second parameter *)
2      (temp) <- COUNT;
3      WHILE (temp != 0)
4      DO
5          IF instruction is SAR or SHR
6          THEN CF <- low-order bit of r/m;
7          FI;
8          IF instruction = SAR
9          THEN r/m <- r/m / 2 (*Signed divide, rounding toward negative infinity*);
10         FI;
11         temp <- temp - 1;
12     OD;
13     IF COUNT = 1
14     THEN
15         IF instruction is SAR
16         THEN OF <- 0;
17         FI;
18     ELSE OF <- undefined;
19     FI;
20

```

sar 是算术右移，高位补原符号位。它对应符号除以 2，并向负无穷取整。

opcode 表接入

```

1  make_group(gp2,
2      EX(rol), EMPTY, EMPTY, EMPTY,
3      EX(shl), EX(shr), EMPTY, EX(sar))
4

```

执行函数实现

```

1  make_EHelper(sar) {
2      uint32_t count = id_src->val & 0x1f;
3      uint32_t bits = id_dest->width * 8;
4      uint32_t result;
5
6      if (count == 0) goto sar_end;
7
8      switch (id_dest->width) {
9          case 1: result = (count >= bits ? ((int8_t)id_dest->val < 0 ? 0xffu : 0u) :
(uint8_t)((int8_t)id_dest->val >> count)); break;
10         case 2: result = (count >= bits ? ((int16_t)id_dest->val < 0 ? 0xffffu : 0u) :
(uint16_t)((int16_t)id_dest->val >> count)); break;
11         case 4: result = (count >= bits ? ((int32_t)id_dest->val < 0 ? 0xffffffffu : 0u) :
(uint32_t)((int32_t)id_dest->val >> count)); break;
12         default: assert(0);
13     }
14
15     rtl_li(&t2, result);
16     operand_write(id_dest, &t2);
17     rtl_update_ZFSF(&t2, id_dest->width);
18     // unnecessary to update CF and OF in NEMU
19     sar_end:
20     print_asm_template2(sar);
21 }
22

```

sar 必须保留符号位,因此不能简单使用无符号右移。实现根据操作数宽度分别把原值转换为 int8_t、int16_t 或 int32_t 后右移,从而利用 C 的有符号右移行为完成符号扩展。若移位次数大于等于位宽,则结果由原符号决定:负数全 1,非负数全 0。结果写回后更新 ZF/SF。

3.1.15 test

i386 手册 Operation

```

1  DEST := LeftSRC AND RightSRC;
2  CF <- 0;
3  OF <- 0;
4

```

test 对两个操作数做按位与，但不保存结果，只根据临时结果更新标志位。它常用于判断某些 bit 是否为 0。

opcode 表接入

```
1 /* 0x84 */ IDEXW(G2E, test, 1), IDEX(G2E, test), EMPTY, EMPTY,
2 /* 0xa8 */ IDEXW(I2a, test, 1), IDEX(I2a, test), EMPTY, EMPTY,
3
```

这部分支持 test r/m, r 以及 test AL/eAX, imm。test r/m, imm 是 group 3 的 /0 扩展：

```
1 make_group(gp3,
2     IDEX(test_I, test), EMPTY, EX(not), EX(neg),
3     EX(mul), EX(imul1), EX(div), EX(idiv))
4
```

gp3 中的 IDEX(test_I, test) 表示进入 group 后还需要继续读取立即数。对应的译码 helper 为：

```
1 make_DHelper(test_I) {
2     decode_op_I(eip, id_src, true);
3 }
4
```

执行函数实现

```
1 make_EHelper(test) {
2     rtl_and(&t2, &id_dest->val, &id_src->val);
3
4     rtl_update_ZFSF(&t2, id_dest->width);
5     rtl_set_CF(&tzero);
6     rtl_set_OF(&tzero);
7
8     print_asm_template2(test);
9 }
10
```

test 的核心计算与 and 相同，都是按位与。区别是 test 不调用 operand_write()，因此临时结果不会写回任何寄存器或内存。函数只用该结果更新 ZF/SF，并按手册要求清零 CF/OF。

3.2 Differential Testing

首先在 nemu/include/common.h 中解注释 #define DIFF_TEST;

然后在 nemu/src/monitor/diff-test/diff-test.c 中修改完善 difftest_step() 的实现逻辑，主要改动包括：

- 给 regcpy_from_nemu() 增加 eflags 同步;
- 在 difftest_step() 中比较 eax/ecx/edx/ebx/esp/ebp/esi/edi/eip ;

- 比较 EFLAGS 中当前 NEMU 实际维护的位：CF、保留位 bit1、ZF、SF、IF、OF；
- 出现差异时用日志打印具体寄存器和数值，并将 nemu_state 置为 NEMU_END。

```

1 void difftest_step(uint32_t eip) {
2     union gdb_regs r;
3     bool diff = false;
4
5     if (is_skip_nemu) {
6         is_skip_nemu = false;
7         return;
8     }
9
10    if (is_skip_qemu) {
11        // to skip the checking of an instruction, just copy the reg state to qemu
12        gdb_getregs(&r);
13        regcpy_from_nemu(r);
14        gdb_setregs(&r);
15        is_skip_qemu = false;
16        return;
17    }
18
19    gdb_si();
20    gdb_getregs(&r);
21
22    // TODO: Check the registers state with QEMU.
23    // Set `diff` as `true` if they are not the same.
24    #define CHECK_REG(name) \
25    do { \
26        if (r.name != cpu.name) { \
27            diff = true; \
28            Log("register %s mismatch after instruction at eip = 0x%08x, qemu = 0x%08x, nemu = 0x%08x", \
29                #name, eip, r.name, cpu.name); \
30        } \
31    } while (0)
32
33    CHECK_REG(eax);
34    CHECK_REG(ecx);
35    CHECK_REG(edx);
36    CHECK_REG(ebx);
37    CHECK_REG(esp);
38    CHECK_REG(ebp);
39    CHECK_REG(esi);
40    CHECK_REG(edi);
41    CHECK_REG(eip);
42
43    #undef CHECK_REG
44

```

```

45     if (diff) {
46         nemu_state = NEMU_END;
47     }
48 }
49

```

3.3 一键回归测试

在 nemu 目录下运行命令 `bash runall.sh` 来批量运行 `nexus-am/tests/cputest/` 中的所有测试，运行结果如下：

```

wyn@ubuntu:~/Documents/ics2017/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!

```

4 PA2-3

4.1 输入输出

实现 in,out 指令： 请实现 in,out 指令，在它们的 helper 函数中分别调用 `pio_read()` 和 `pio_write()` 函数。由于 NEMU 中有一些设备的行为是我们自定义的，与 QEMU 中的标准设备的行为不完全一样（例如 NEMU 中的串口总是就绪的，但 QEMU 中的串口并不是这样），这导致在 NEMU 中执行 in 和 out 指令今的结果与 QEMU 可能会存在不可调整的偏差。为了使得 differential testing 可以正常工作，我们在这两条指令中调用了相应的函数来设置 `is_skip_qemu` 标志，来跳过与 QEMU 的检查。

实现后，在 `nexus-am/arch/x86-nemu/src/trm.c` 中定义宏 `HAS_SERIAL`，然后在 `nexus-am/apps/hello` 目录下键入 `make run`，在 NEMU 中运行基于 AM 的 hello 程序。如果你的实现正确，你将会看到程序往终端输出了 10 行 Hello World!

实现过程 .

- 在 system.c 里实现了 in/out 的 helper:
 - in 调用 pio_read(), 把结果写回累加器操作数;
 - out 调用 pio_write();
 - in 调用 pio_read(), 把结果写回累加器操作数;
 - out 调用 pio_write();
 - 两条指令都保留了 diff_test_skip_qemu(), 用于跳过和 QEMU 的比较;
- 在 exec.c 的 opcode_table 中补上了 8 个 I/O opcode;
- 在 all-instr.h 中补了 make_EHelper(in) 和 make_EHelper(out) 声明;
- 在 nexus-am/am/arch/x86-nemu/src/trm.c 中启用了 HAS_SERIAL;

```

1  make_EHelper(in) {
2      //TODO();
3      rtl_li(&t0, pio_read(id_src->val, id_dest->width));
4      operand_write(id_dest, &t0);
5
6      print_asm_template2(in);
7
8      #ifdef DIFF_TEST
9      diff_test_skip_qemu();
10     #endif
11 }
12
13 make_EHelper(out) {
14     //TODO();
15     pio_write(id_dest->val, id_src->width, id_src->val);
16
17     print_asm_template2(out);
18
19     #ifdef DIFF_TEST
20     diff_test_skip_qemu();
21     #endif
22 }
23

```

```

1  /* 0xe4 */ IDEXW(in_I2a, in, 1), IDEX(in_I2a, in), IDEXW(out_a2I, out, 1), IDEX(out_a2I,
2  out),
3  ...
4  /* 0xec */ IDEXW(in_dx2a, in, 1), IDEX(in_dx2a, in), IDEXW(out_a2dx, out, 1), IDEX(out_a2dx,
5  out),
6

```

运行 **Hello World** 在 nexus-am/apps/hello 目录下键入 make run, 运行结果如下:

```
wyn@ubuntu:~/Documents/ics2017/nexus-am/apps/hello$ make run
Building hello [native]
make[1]: Entering directory '/home/wyn/Documents/ics2017/nexus-am'
make[2]: Entering directory '/home/wyn/Documents/ics2017/nexus-am/am'
Building am [native]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/wyn/Documents/ics2017/nexus-am/am'
make[1]: Leaving directory '/home/wyn/Documents/ics2017/nexus-am'
make[1]: Entering directory '/home/wyn/Documents/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/wyn/Documents/ics2017/nexus-am/libs/klib'
/home/wyn/Documents/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
```

4.2 时钟

实现 IOE: 实现 `_uptime()` 后, 在 NEMU 中运行 `timetest` 程序 (在 `nexus-am/tessts/timetest` 目录下, 编译和运行方式请参考 PA2 第三阶段指导书, 此后不再额外说明). 如果你的实现正确, 你将会看到程序每隔 1 秒输出一句话.

实现过程

改动很简单:

- 保留 `_ioe_init()` 在启动时读取 `RTC_PORT`;
- 实现 `_uptime()` 返回 `inl(RTC_PORT) - boot_time`

也就是把 NEMU 定时器设备提供的“当前毫秒数”转换成“系统启动后的毫秒数”, 符合 AM 规范。

在 `nexus-am/am/arch/x86-nemu/src/ioe.c` 中:

```
1 void _ioe_init() {
2     boot_time = inl(RTC_PORT);
3 }
4
5 unsigned long _uptime() {
6     return inl(RTC_PORT) - boot_time;
7 }
8
```

完成后在 `nexus-am/tests/timetest` 目录下输入命令 `make ARCH=x86-nemu run`:

```
[src/monitor/diff-test/diff-test.c,101,init_difftest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nexus-am/tests/timetest/build/timetest-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 04:09:31, Apr 25 2026
For help, type "help"
(nemu) c
1 second.
2 seconds.
3 seconds.
4 seconds.
5 seconds.
6 seconds.
7 seconds.
8 seconds.
9 seconds.
10 seconds.
11 seconds.
12 seconds.
13 seconds.
```

看看 nemu 跑得多快。

注释掉 nemu/include/common.h 中的 DEBUG 和 DIFF_TEST 宏, 在 nexus-am/apps/ 各个子目录下运行 make ARCH=x86-nemu run 进行 benchmark 跑分:

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nexus-am/apps/dhrystone/build/dhrystone-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:11:35, Apr 25 2026
For help, type "help"
(nemu) c
Dhrystone Benchmark, Version C, Version 2.2
Trying 500000 runs through Dhrystone.
Finished in 18129 ms
=====
Dhrystone PASS          56 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x0010006e
```

图 4.1: Dhrystone

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nexus-am/apps/coremark/build/coremark-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:11:35, Apr 25 2026
For help, type "help"
(nemu) c
Running CoreMark for 1000 iterations
2K performance run parameters for coremark.
CoreMark Size      : 666
Total time (ms)    : 19593
Iterations         : 1000
Compiler version   : GCC5.4.0 20160609
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0xd340
Finished in 19593 ms.
=====
CoreMark PASS      228 Marks
                  vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x0010006e
```

图 4.2: Coremark

```

Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:11:35, Apr 25 2026
For help, type "help"
(nemu) c
[qsrt] Quick sort: * Passed.
      min time: 1458 ms [378]
[queen] Queen placement: * Passed.
      min time: 2083 ms [247]
[bf] Brainf**k interpreter: * Passed.
      min time: 12450 ms [210]
[fib] Fibonacci number: * Passed.
      min time: 28343 ms [100]
[sieve] Eratosthenes sieve: * Passed.
      min time: 22025 ms [192]
[15pz] A* 15-puzzle search: * Passed.
      min time: 4441 ms [130]
[dinic] Dinic's maxflow algorithm: * Passed.
      min time: 3979 ms [340]
[lzip] Lzip compression: * Passed.
      min time: 10664 ms [248]
[ssort] Suffix sort: * Passed.
      min time: 2188 ms [270]
[md5] MD5 digest: * Passed.
      min time: 21273 ms [92]
=====
MicroBench PASS          220 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x00100032

```

图 4.3: microbench-ref

```

Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:11:35, Apr 25 2026
For help, type "help"
(nemu) c
[qsrt] Quick sort: * Passed.
[queen] Queen placement: * Passed.
[bf] Brainf**k interpreter: * Passed.
[fib] Fibonacci number: * Passed.
[sieve] Eratosthenes sieve: * Passed.
[15pz] A* 15-puzzle search: * Passed.
[dinic] Dinic's maxflow algorithm: * Passed.
[lzip] Lzip compression: * Passed.
[ssort] Suffix sort: * Passed.
[md5] MD5 digest: * Passed.
=====
MicroBench PASS
nemu: HIT GOOD TRAP at eip = 0x00100032

```

图 4.4: microbench-test

4.3 键盘

实现 IOE(2): 实现 `__read_key()` 后, 在 NEMU 中运行 `keytest` 程序 (在 `nexus-am/tests/keytest` 目录下). 如果你的实现正确, 在程序运行时弹出的新窗口中按下按键, 你将会看到程序输出相应的按键信息.

实现过程

修改 `nexus-am/am/arch/x86-nemu/src/ioe.c`, 主要修改内容:

- 增加 `i8042` 相关端口常量: `0x60` 数据端口、`0x64` 状态端口、`0x1` 有键标志;
- 实现 `__read_key()`:
 - 先读 `0x64` 判断是否有按键
 - 没有则返回 `__KEY_NONE`
 - 有的话再从 `0x60` 读出按键编码并返回.

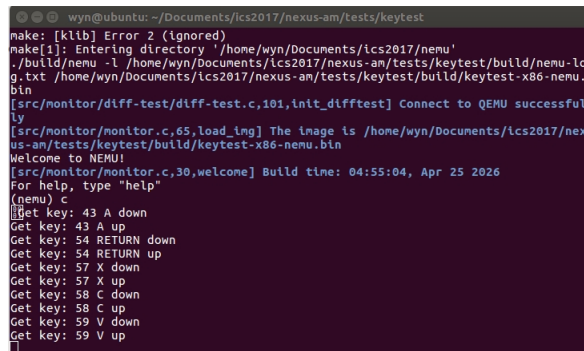
这样和 NEMU 里的 keyboard.c 是对上的：状态口负责把队列头准备到数据口，数据口读走后会清状态位。

```

1  #define I8042_DATA_PORT 0x60
2  #define I8042_STATUS_PORT 0x64
3  #define I8042_STATUS_HASKEY_MASK 0x1
4  ...
5  int _read_key() {
6      if ((inb(I8042_STATUS_PORT) & I8042_STATUS_HASKEY_MASK) == 0) {
7          return _KEY_NONE;
8      }
9      return inl(I8042_DATA_PORT);
10 }
11

```

完成后，在 nexus-am/tests/keytest 目录下运行 make ARCH=x86-nemu run，在弹出的新窗口中按下键盘会被捕捉：



```

wyn@ubuntu: ~/Documents/ics2017/nexus-am/tests/keytest
make: [klb] Error 2 (ignored)
make[1]: Entering directory '/home/wyn/Documents/ics2017/nemu'
./build/nemu -l /home/wyn/Documents/ics2017/nexus-am/tests/keytest/build/nemu-log.txt /home/wyn/Documents/ics2017/nexus-am/tests/keytest/build/keytest-x86-nemu.bin
[src/monitor/diff-test/diff-test.c,101,init_difftest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nexus-am/tests/keytest/build/keytest-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 04:55:04, Apr 25 2026
For help, type "help"
(nemu) C
Get key: 43 A down
Get key: 43 A up
Get key: 54 RETURN down
Get key: 54 RETURN up
Get key: 57 X down
Get key: 57 X up
Get key: 58 C down
Get key: 58 C up
Get key: 59 V down
Get key: 59 V up

```

4.4 VGA

添加内存映射 I/O： 在 paddr_read() 和 paddr_write() 中加入对内存映射 I/O 的判断. 通过 is_mmio() 函数判断一个物理地址是否被映射到 I/O 空间, 如果是, is_mmio() 会返回映射号, 否则返回-1. 内存映射 I/O 的访问需要调用 mmio_read() 或 mmio_write(), 调用时需要提供映射号. 如果不是内存映射 I/O 的访问, 就访问 pmem. 实现后, 在 NEMU 中运行 videotest 程序 (在 nexus-am/tests/videotest 目录下). 如果内存映射 I/O 实现正确, 你会看到新窗口中输出了一些颜色信息.

实现过程

修改 nemu/src/memory/memory.c，主要修改内容：

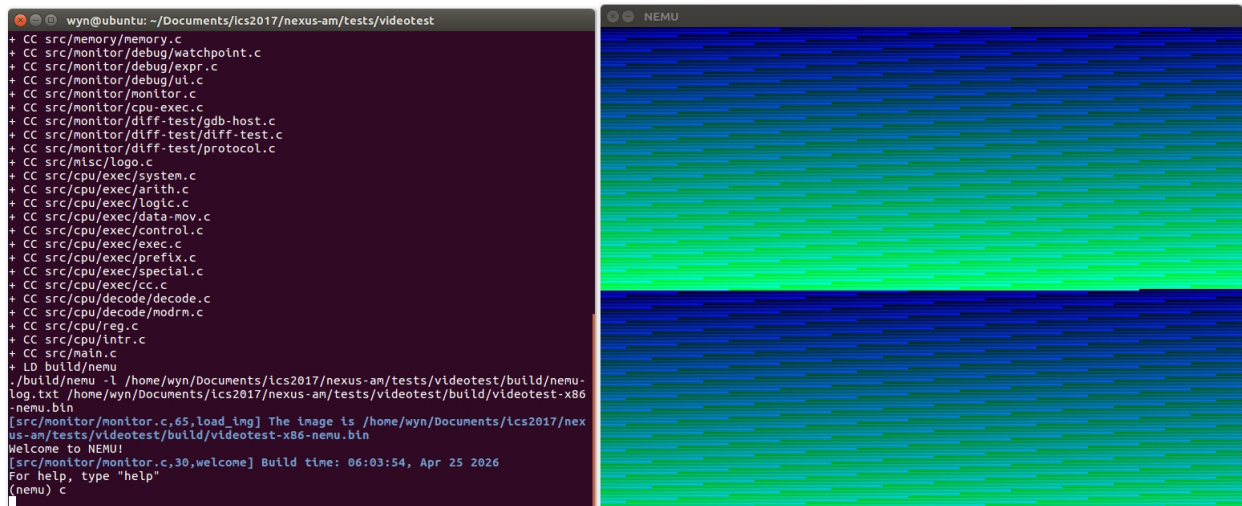
- 引入 device/mmio.h;
- 在 paddr_read() 中先调用 is_mmio(addr): 如果返回映射号, 则调用 mmio_read(addr, len, map_NO); 否则按原来方式访问 pmem;
- 在 paddr_write() 中同样先判断: 如果是 MMIO, 则调用 mmio_write(addr, len, data, map_NO); 否则写回 pmem.

```

1  uint32_t paddr_read(paddr_t addr, int len) {
2      int map_NO = is_mmio(addr);
3      if (map_NO != -1) {
4          return mmio_read(addr, len, map_NO);
5      }
6      return pmem_rw(addr, uint32_t) & (-0u >> ((4 - len) << 3));
7  }
8
9  void paddr_write(paddr_t addr, int len, uint32_t data) {
10     int map_NO = is_mmio(addr);
11     if (map_NO != -1) {
12         mmio_write(addr, len, data, map_NO);
13     }
14     return;
15     memcpy(guest_to_host(addr), &data, len);
16 }
17

```

完成后，在 nexus-am/tests/videotest 目录下运行，结果如下：



实现 IOE(3): 事实上，刚才输出的颜色信息并不是 videotest 输出的画面，这是因为框架代码中的 `_draw_rect()` 并未正确实现其功能。你需要实现正确的 `_draw_rect()`。实现后，在 NEMU 中重新运行 videotest。如果你的实现正确，你将会看到新窗口中输出了相应的动画效果。

实现过程

修改 nexus-am/am/arch/x86-nemu/src/ioe.c，主要修改内容：

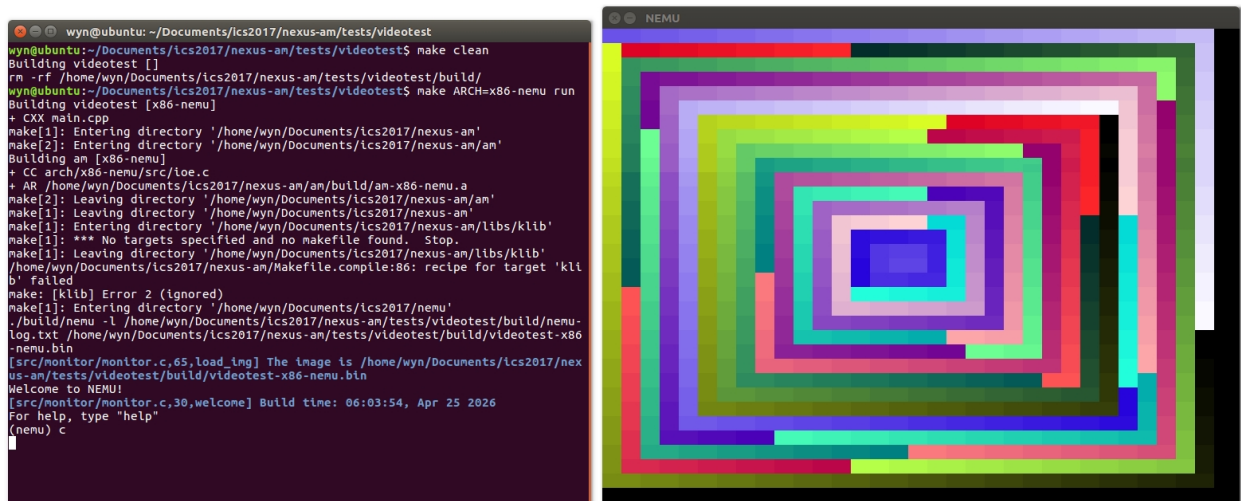
- 给 `_draw_rect()` 按正确语义实现了矩形绘制；
- 逐行把 pixels 中按行存储的 $w * h$ 像素复制到显存 fb；
- 加了基本边界裁剪，避免写出屏幕范围；
- 不再像之前那样把整块显存填成测试图案；

```

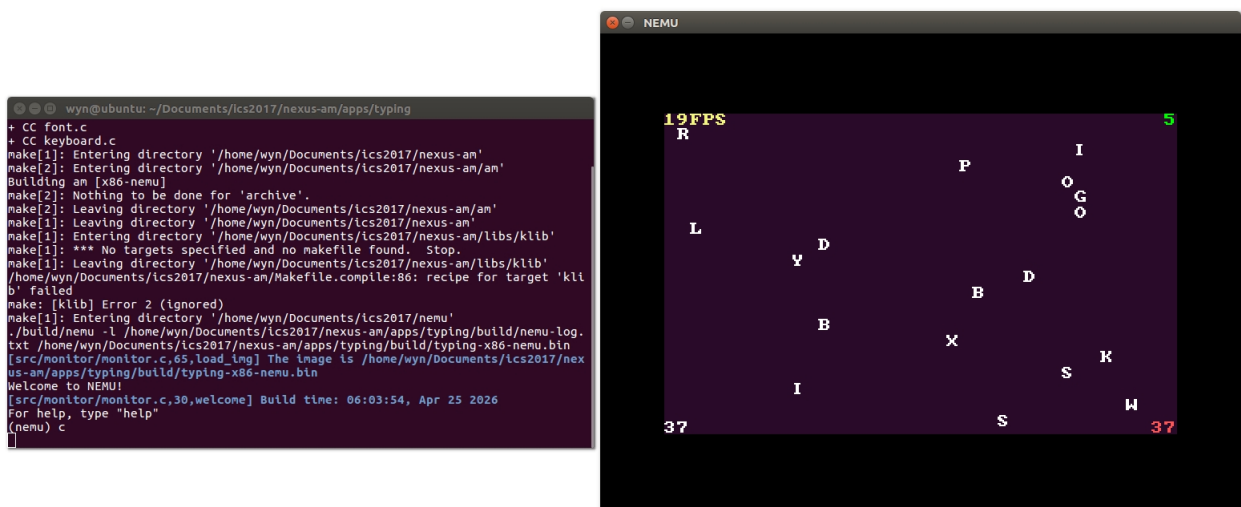
1 void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h) {
2     if (w <= 0 || h <= 0 || x >= _screen.width || y >= _screen.height) {
3         return;
4     }
5
6     int cp_bytes = sizeof(uint32_t) * min(w, _screen.width - x);
7     for (int j = 0; j < h && y + j < _screen.height; j++) {
8         memcpy(&fb[(y + j) * _screen.width + x], pixels, cp_bytes);
9         pixels += w;
10    }
11 }
12

```

现在 videotest 通过 `_draw_rect()` 写到 0x40000 这片 MMIO framebuffer, 此时再在 `nexus-am/tests/videotest` 目录下运行, 发现可以显示真正的动画内容:



运行打字小游戏 在 `nexus-am/apps/typing` 目录下输入命令 `make ARCH=x86-nemu run`:



可见成功运行了打字小游戏, 界面上不断下落字母符号, 按下键盘对应字母可以消除该字母同时

右上角绿色计分数增加，没有及时消除的字母符号落到界面底部则导致右下角红色计数增加。运行成功。

5 思考题与必答题

5.1 思考题

1. 立即数背后的故事。 Motorola68k 系列的处理器都是大端架构的。现在问题来了。考虑以下两种情况：

- 假设我们需要将 NEMU 运行在 Motorola 68k 的机器上 (把 NEMU 的源代码编译成 Motorola 68k 的机器码)；
- 假设我们需要编写一个新的模拟器 NEMU-Motorola-68k, 模拟器本身运行在 x86 架构中, 但它模拟的是 Motorola 68k 程序的执行；

在这两种情况下, 你需要注意些什么问题? 为什么会产生这些问题? 怎么解决它们? 事实上不仅仅是立即数的访问, 长度大于 1 字节的内存访问都需要考虑类似的问题。

答: 问题的本质是: 模拟器中至少同时存在两个字节序概念, 一个是宿主机字节序, 一个是客户机字节序。宿主机是运行 NEMU 的真实机器; 客户机是 NEMU 正在模拟的 ISA。instr_fetch(eip, 4) 读取的是客户机内存中的 4 个字节, 但把这 4 个字节解释成整数时, 不能无条件依赖宿主机 CPU 的解释方式。

如果把现在的 NEMU 源码编译到 Motorola 68k 机器上运行, 而 NEMU 仍然模拟 x86 程序, 那么宿主机变成大端, 客户机仍然是小端。x86 指令流中的立即数 34 12 00 00 表示 0x00001234, 但如果代码只是把这 4 个字节直接拷贝进 uint32_t 并让 68k 按大端解释, 就会得到 0x34120000, 立即数错误。解决方法是让 instr_fetch() 或底层 vaddr_read() 明确按客户机字节序组装整数, 例如对 x86:

```
1  uint32_t val = 0;
2  for (int i = 0; i < len; i++) {
3      val |= ((uint32_t)vaddr_read(eip + i, 1)) << (8 * i);
4  }
5
```

这样无论宿主机是大端还是小端, x86 客户机读出的多字节值都按小端解释。

如果编写一个运行在 x86 上的 NEMU-Motorola-68k, 情况正好反过来: 宿主机是小端, 但被模拟的 Motorola 68k 是大端。68k 程序中的 32 位立即数 0x1234 在客户机内存中应当是 00 00 12 34。如果模拟器直接让 x86 宿主机按小端解释这 4 个字节, 就会得到 0x34120000。解决方法同样是把客户机字节序写进访存接口, 例如对 68k:

```
1  uint32_t val = 0;
2  for (int i = 0; i < len; i++) {
3      val = (val << 8) | vaddr_read(eip + i, 1);
4  }
5
```

所以, 模拟器不能把“从内存读出若干字节”和“解释成整数”混为一谈。所有长度大于 1 字节的访存, 包括立即数、指令读取、数据读写、栈访问、设备寄存器访问, 都应该通过统一的接口处理客

户机字节序。这样才能保证模拟结果不依赖宿主机字节序。

2. 堆和栈在哪里？ 我们知道代码和数据都在可执行文件里面，但却没有提到堆 (heap) 和栈 (stack)。为什么堆和栈的内容没有放入可执行文件里面？那程序运行时刻用到的堆和栈又是怎么来的？AM 的代码是否能给你带来一些启发？

答：可执行文件保存的是程序开始运行前已经确定的内容，例如代码段、只读数据、已初始化全局变量，以及未初始化全局变量的大小描述。堆和栈不放在可执行文件里，是因为它们的内容不是静态确定的。

栈保存函数调用过程中的返回地址、保存的寄存器、局部变量和临时数据。这些内容依赖程序运行路径：不同输入、不同递归深度、不同函数调用顺序都会产生不同的栈内容。堆由 malloc 等动态分配接口按运行时需求申请，大小和内容也取决于程序运行过程。因此可执行文件最多描述“程序需要哪些静态段”和“入口地址在哪里”，不会把运行时才产生的堆栈内容提前保存进去。

程序运行时，堆和栈由运行环境创建。普通操作系统会在加载 ELF 后建立进程地址空间，映射代码段、数据段，设置用户栈，再把控制权交给程序入口。堆通常从数据段之后开始，由运行库通过 brk/sbrk 或 mmap 等系统调用扩展。

AM 的代码给出的启发是：在没有完整操作系统时，运行时环境也必须自己准备栈。nexus-am/am/arch/x86-nemu/img/boot/start.S 这类启动代码会在进入 C 程序前设置栈指针；链接脚本会安排程序各段的位置。也就是说，堆和栈不是凭空出现的，它们来自 loader、运行库、操作系统或 AM 启动代码对内存布局的约定和初始化。

3. 捕捉死循环。 NEMU 除了作为模拟器之外，还具有简单的调试功能，可以设置断点，查看程序状态。如果让你为 NEMU 添加如下功能：

当用户程序陷入死循环时，让用户程序暂停下来，并输出相应的提示信息。

你觉得应该如何实现？如果你感到疑惑，在互联网上搜索相关信息。

答：要让 NEMU 在用户程序陷入死循环时暂停，关键是定义“死循环”的可观测特征。完全精确地判断任意程序是否死循环等价于停机问题，理论上不可判定。因此模拟器只能实现启发式检测。

一种直接方法是在 cpu_exec() 的执行循环中记录最近一段时间的 EIP 轨迹。如果发现同一个短循环被重复执行很多次，并且在这段时间内没有发生设备输入、内存状态变化、系统调用或 nemu_trap，就认为程序可能陷入死循环。例如维护一个环形缓冲区，记录最近 N 条指令的 EIP；当某个 EIP 或某个 EIP 序列重复超过阈值时暂停执行并提示用户。

更稳妥的做法是结合基本块和状态摘要：

- 记录每次跳转后的 EIP，识别循环头。
- 对循环执行次数计数。
- 记录循环期间是否有内存写、I/O 访问、中断、键盘输入等外部可见行为。
- 如果同一循环执行次数超过阈值，且没有外部可见行为，就暂停 NEMU。

伪代码可以写成：

```
1  if (cpu.eip == last_loop_head && no_observable_event) {
2      loop_count++;
3      if (loop_count > LOOP_THRESHOLD) {
4          nemu_state = STOP;
```

```
5     printf("Possible infinite loop at EIP = 0x%x\n", cpu.eip);
6 }
7 } else {
8     update_loop_detector(cpu.eip);
9 }
10
```

不过, 这种实现可能误报。例如操作系统 idle loop 本来就是循环等待中断; 游戏主循环也可能长时间重复执行。因此检测逻辑应当允许用户开关, 也可以只在 PA 调试时启用。更强的调试方式是结合 watchpoint、断点、指令计数上限和 trace, 先暂停再让用户判断。

4. 理解 volatile 关键字. 也许你从来都没听说过 C 语言中有 volatile 这个关键字, 但它从 C 语言诞生开始就一直存在。volatile 关键字的作用十分特别, 它的作用是避免编译器对相应代码进行优化。你或许会感到疑惑, 代码优化不是一件好事情吗? 为什么会有 volatile 这种奇葩的存在? 思考一下, 如果代码中的地址 0x8049000 最终被映射到一个设备寄存器, 去掉 volatile 可能会带来什么问题?

答: volatile 可以告诉编译器: 这个对象的值可能被当前程序流之外的因素改变, 每次读写都必须真的发生, 不能随便缓存、合并或删除。

题目中的代码访问固定地址 0x8049000:

```
1 void fun() {
2     volatile unsigned char *p = (void *)0x8049000;
3     *p = 0;
4     while (*p != 0xff);
5     *p = 0x33;
6     *p = 0x34;
7     *p = 0x86;
8 }
9
```

保留 volatile 时, while (*p != 0xff) 中的每次循环都必须重新从地址 0x8049000 读一次, 因为这个地址的值可能被硬件设备改变。后面的三次写入 0x33、0x34、0x86 也都必须按顺序发出。

去掉 volatile 后, 编译器会按普通内存优化。它可能认为 *p = 0 后, 在当前函数内部没有别的代码能把 *p 改成 0xff, 于是 while (*p != 0xff) 可以被优化成死循环; 后面的写入永远不可达。也可能把连续写入合并, 只保留最后一次写入 0x86, 因为对普通内存来说, 中间的 0x33 和 0x34 没有被读取, 似乎没有意义。

如果 0x8049000 映射的是设备寄存器, 去掉 volatile 就会出错。设备寄存器的读写有副作用: 读一次可能代表获取设备状态, 写一次可能代表向设备发送命令。连续写 0x33、0x34、0x86 可能是给设备发送三个命令字节, 任何一次写被删除或重排都会改变硬件行为。因此访问 MMIO、设备寄存器、中断共享变量时需要 volatile, 有时还需要内存屏障保证顺序。

5. 如何检测多个键同时被按下. 在游戏中, 很多时候需要判断玩家是否同时按下了多个键, 例如 RPG 游戏中的八方向行走, 格斗游戏中的组合招式等等。根据键盘码的特性, 你知道这些功能是如何实现的吗?

答: 键盘事件通常分为按下和释放。以 PC 键盘扫描码为例, 一个键按下时产生 make code, 释放时产生 break code。要判断多个键是否同时按下, 程序不能只看“当前收到的一个键盘码”, 而要维

维护一个键盘状态表。

做法是为每个键维护一个布尔状态：

```
1 bool key_down[KEY_MAX];
2
```

收到按下事件时：

```
1 key_down[key] = true;
2
```

收到释放事件时：

```
1 key_down[key] = false;
2
```

游戏逻辑每一帧检查状态表。例如八方向行走可以同时检查：

```
1 if (key_down[KEY_UP] && key_down[KEY_LEFT]) {
2     move_up_left();
3 }
4
```

格斗游戏的组合键也是类似原理，只是还会记录按键时间序列。例如在一个短时间窗口内依次出现“下、前、拳”，就识别为招式。多个键同时按下的核心不是键盘一次发来多个码，而是程序保存每个键当前是否处于按下状态。

6. 神奇的调色板。 在一些 90 年代的游戏里，很多渐出渐入效果都是通过调色板实现的，聪明的你知道其中的玄机吗？

答：8 位颜色深度下，显存中每个像素不是直接保存 RGB 值，而是保存一个 0 到 255 的索引。真正的颜色在调色板中：

```
1 pixel_value -> palette[pixel_value] -> RGBA
2
```

因此，同一张 8 位图像在显存中的像素值不变，只要改变调色板，屏幕显示出来的颜色就会整体改变。

90 年代游戏里的渐入渐出常利用这一点。渐出时，不需要修改整张图片的每个像素，只要逐步把调色板中每个颜色的 RGB 分量向黑色靠近：

```
1 R <- R * k
2 G <- G * k
3 B <- B * k
4
```

其中 k 从 1.0 逐渐减小到 0。渐入则反过来，从全黑调色板逐渐恢复到原始调色板。因为调色板只有 256 项，每项 4 字节，修改一次只需要处理大约 1KB 数据，比逐像素修改整屏图像快得多。

同理，闪白、中毒变绿、昼夜变化、水波色彩变化等效果，也可以通过替换或插值调色板实现。这就是调色板动画的关键：像素索引不变，颜色解释表变化。

5.2 必答题

1. 在 `nemu/include/cpu/rtl.h` 中, 你会看到由 `static inline` 开头定义的各种 RTL 指令函数. 选择其中一个函数, 分别尝试去掉 `static`, 去掉 `inline` 或去掉两者, 然后重新进行编译, 你会看到发生错误. 请分别解释为什么会发生这些错误? 你有办法证明你的想法吗?

答: `nemu/include/cpu/rtl.h` 中有大量 RTL 函数, 例如:

```
1 static inline void rtl_li(rtlreg_t* dest, uint32_t imm) {
2     *dest = imm;
3 }
4
```

这些函数定义在头文件里, 而不是.c 文件里。头文件会被很多.c 文件包含, 所以同一份函数定义会出现在多个编译单元中。`static inline` 同时解决两个问题:

- `static` 让函数具有内部链接, 每个.c 文件得到自己的局部版本, 不会在链接阶段和其他.o 文件里的同名函数冲突。
- `inline` 建议编译器把函数体直接展开到调用处, 减少大量小函数调用开销, 也避免未使用的静态函数产生告警。

如果只去掉 `static`, 变成:

```
1 inline void rtl_li(rtlreg_t* dest, uint32_t imm) { ... }
2
```

函数具有外部链接。根据 C `inline` 语义和 GCC 选项, 可能出现两类问题: 一种是链接器发现多个编译单元都提供了同名外部定义, 报 `multiple definition`; 另一种是编译器没有为某个 `inline` 函数生成真正的外部定义, 而其他地方又需要调用它, 导致 `undefined reference`。根本原因都是: 头文件中的外部链接函数定义被多个.c 文件重复看到, 却没有一个单独.c 文件负责提供唯一的外部定义。

如果只去掉 `inline`, 变成:

```
1 static void rtl_li(rtlreg_t* dest, uint32_t imm) { ... }
2
```

链接冲突不会发生, 因为仍然是内部链接。但每个包含 `rtl.h` 的.c 文件都会生成一批 `static` 函数定义, 而其中很多函数在该.c 文件内并没有被使用。由于 `Makefile` 中启用了 `-Wall -Werror`, GCC 会把 `defined but not used` 这类警告当成错误, 编译失败。

如果同时去掉 `static` 和 `inline`, 变成:

```
1 void rtl_li(rtlreg_t* dest, uint32_t imm) { ... }
2
```

这是最典型的错误: 普通外部函数被定义在头文件中, 每个包含它的.c 文件都会生成一个全局符号。编译单个.c 可能通过, 但链接多个.o 时会出现同名符号的 `multiple definition`。

2. 了解 `Makefile` 请描述你在 `nemu` 目录下敲入 `make` 后, `make` 程序如何组织.c 和.h 文件, 最终生成可执行文件 `nemu/build/nemu`。(这个问题包括两个方面: `Makefile` 的工作方式和编译链接的过程。) 关于 `Makefile` 工作方式的提示:

- Makefile 中使用了变量, 包含文件等特性;
- Makefile 运用并重写了一些 implicit rules;
- 在 manmake 中搜索 -n 选项, 也许对你有帮助;
- RTEM。

答: 在 nemu/ 目录下敲入 make 后, 默认目标由以下语句决定:

```
1 .DEFAULT_GOAL = app
2
```

所以 make 会尝试构建 app。app 的依赖是:

```
1 app: $(BINARY)
2
```

其中变量定义为:

```
1 NAME = nemu
2 INC_DIR += ./include
3 BUILD_DIR ?= ./build
4 OBJ_DIR ?= $(BUILD_DIR)/obj
5 BINARY ?= $(BUILD_DIR)/$(NAME)
6
```

因此最终目标是 build/nemu。

Makefile 先包含 Makefile.git:

```
1 include Makefile.git
2
```

这个文件定义了 git_commit 宏。编译和运行目标会调用它自动提交记录, 这是 PA 框架用来记录实验过程的机制。

源文件列表由:

```
1 SRCS = $(shell find src/ -name "*.c")
2
```

生成。\$(shell ...) 会在解析 Makefile 时执行 shell 命令, 找到 src/ 目录下所有 .c 文件。然后:

```
1 OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)
2
```

把每个源文件路径替换成目标文件路径。例如:

```
1 src/cpu/exec/arith.c -> build/obj/cpu/exec/arith.o
2
```

编译 .c 到 .o 使用模式规则:

```

1 $(OBJ_DIR)/%.o: src/%.c
2     @echo + CC $<
3     @mkdir -p $(dir $@)
4     @$ (CC) $(CFLAGS) -c -o $@ $<
5

```

这条规则重写了 make 默认的 C 编译 implicit rule。\$< 表示第一个依赖文件，也就是源文件；\$@ 表示目标文件。mkdir -p \$(dir \$@) 先创建目标文件所在目录，然后 gcc -c 只编译不链接，生成.o 文件。

编译选项为：

```

1 CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)
2

```

其中 \$(INCLUDES) 来自：

```

1 INCLUDES = $(addprefix -I, $(INC_DIR))
2

```

会展开成 -I./include，使得 #include "cpu/rtl.h" 这类头文件可以被找到。-MMD 会让 GCC 生成.d 依赖文件，记录.c 文件依赖了哪些.h 文件。Makefile 通过：

```

1 -include $(OBJS:.o=.d)
2

```

把这些依赖文件包含进来。前面的 - 表示这些.d 文件不存在时不报错。这样如果某个头文件变化，make 能知道哪些.o 需要重新编译。

所有.o 生成后，链接目标为：

```

1 $(BINARY): $(OBJS)
2     $(call git_commit, "compile")
3     @echo + LD $@
4     @$ (LD) -O2 -o $@ $^ -lsdl2 -lreadline
5

```

\$. 表示所有依赖，也就是全部目标文件。链接命令把所有.o 和 SDL2、readline 库链接成最终可执行文件 build/nemu。

因此完整流程是：

```

1 make
2 -> 默认目标 app
3 -> app 依赖 build/nemu
4 -> build/nemu 依赖所有 build/obj/**/*.o
5 -> 每个 .o 由对应 src/**/*.c 编译得到
6 -> 编译时读取 include/ 下的 .h，并生成 .d 依赖文件
7 -> 所有 .o 链接成 build/nemu
8

```